Model Context Protocol

定义

MCP:Model Context Protocol

LLM: Large Language Model（大型语言模型）

Anthropic：Claude AI的公司

Function Calling：AI模型调用函数的机制

什么是MCP?

由 Anthropic 推出的一种开放标准协议，目的是统一LLM与外部数据源和工具之间的通信协议。

简单来说MCP就像USB一样，兼具双向通信和通用。

MCP使用场景

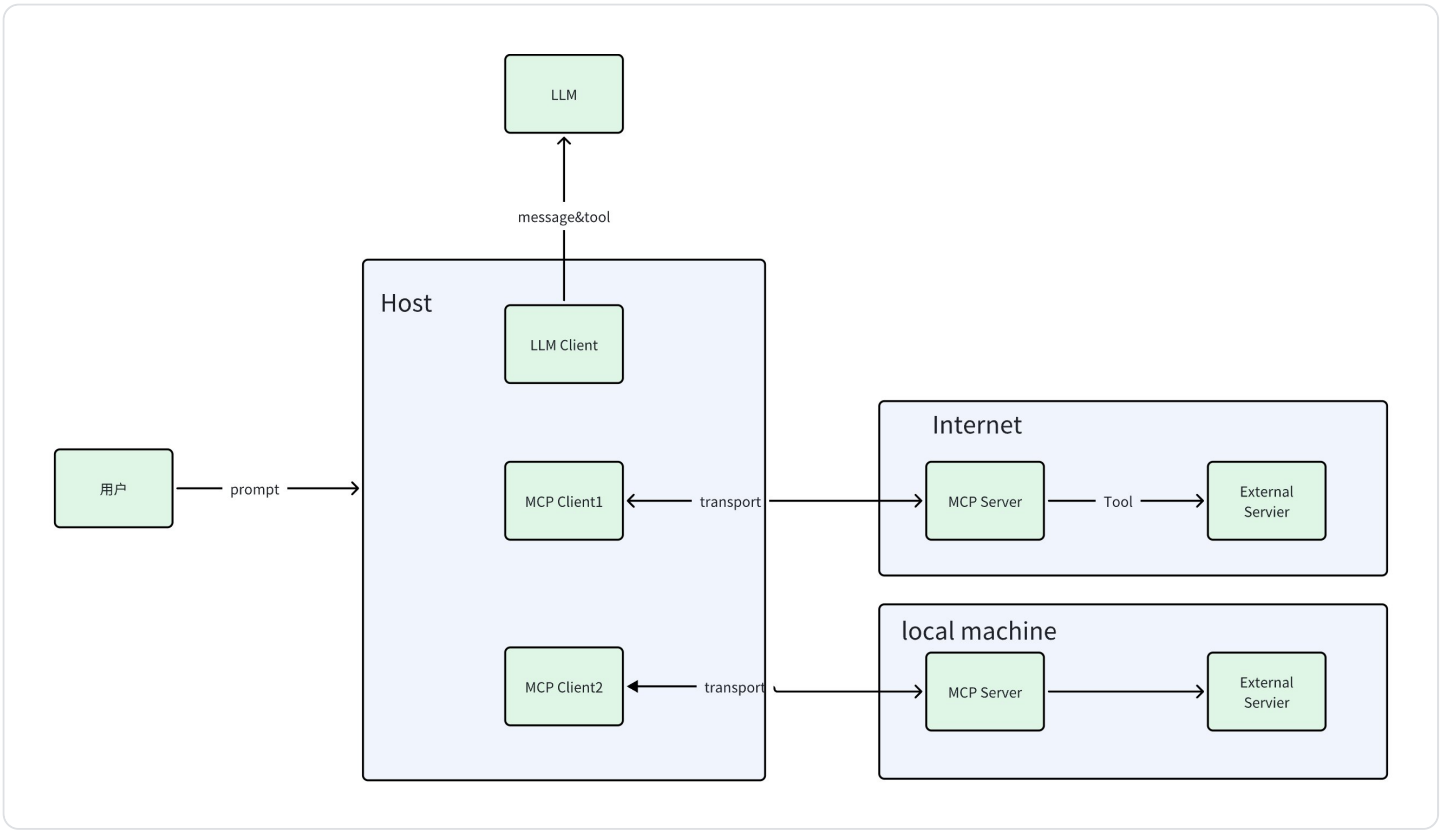
目前协议更倾向于在ide中使用，用于给LLM提供功能列表（Tools,Prompts,resources,Utilities），LLM会自行选择调用

MCP 核心架构

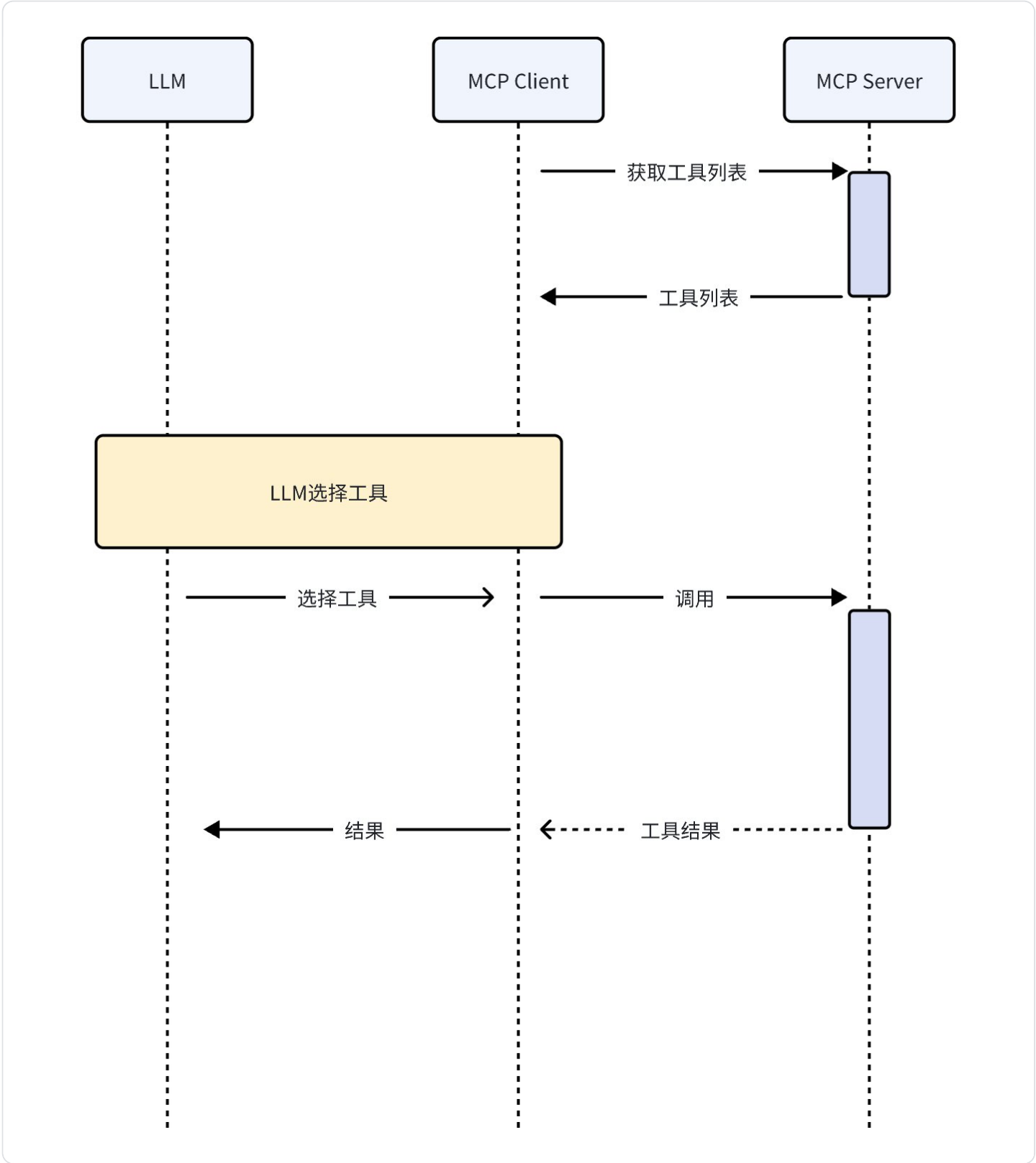
Client :与对应Sever交互的本地客户端，注意client和Server是一一对应的，Client 和Server是双向传输

Server:提供Tools,Prompts,resources,Utilities 四大类型服务

Transport: Client和Server的传输机制


LLM
message&tool
Host
LLM Client
用户
prompt
MCP Client1
transport
Internet
MCP Server
Tool
External Servier
local machine
MCP Client2
transport
MCP Server
External Servier

MCP 调用流程



基础协议

Message： 基于JSON-RPC 2.0的通信消息结构

Requests

Requests are sent from the client to the server or vice versa.

```
{
  jsonrpc: "2.0";
  id: string | number;
  method: string;
  params?: {
    [key: string]: unknown;
  };
}
```

- Requests **MUST** include a string or integer ID.
- Unlike base JSON-RPC, the ID **MUST NOT** be `null`.
- The request ID **MUST NOT** have been previously used by the requestor within the same session.

Responses

Responses are sent in reply to requests.

```
{
  jsonrpc: "2.0";
  id: string | number;
  result?: {
    [key: string]: unknown;
  }
  error?: {
    code: number;
    message: string;
    data?: unknown;
  }
}
```

- Responses **MUST** include the same ID as the request they correspond to.
- Either a `result` or an `error` **MUST** be set. A response **MUST NOT** set both.
- Error codes **MUST** be integers.

Notifications

Notifications are sent from the client to the server or vice versa. They do not expect a response.

```
{
  jsonrpc: "2.0";
  method: string;
  params?: {
    [key: string]: unknown;
  };
}
```

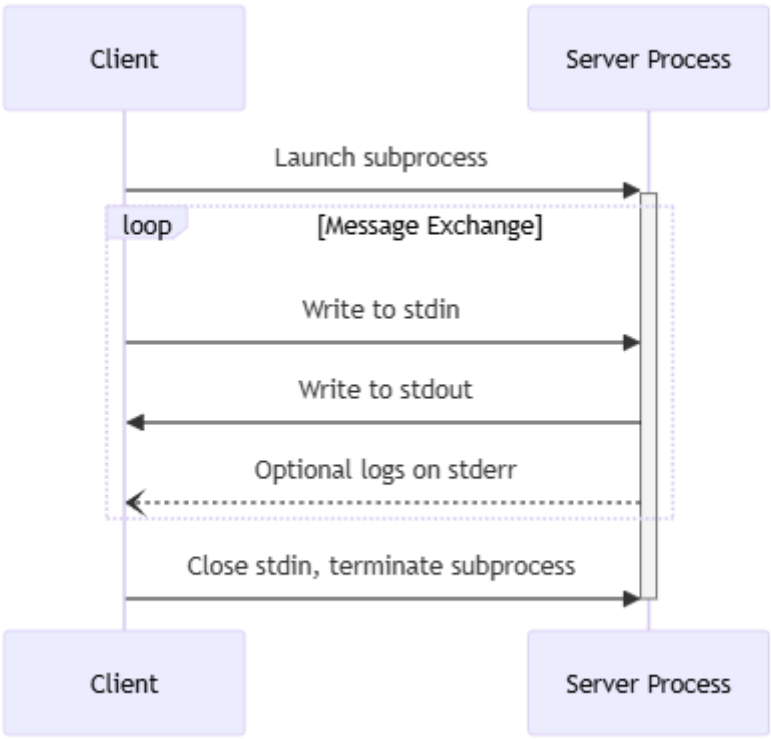
- Notifications **MUST NOT** include an ID.

Transports：传输机制

Client和Server目前有个2种传输机制：

stdio：通过标准输入和标准输出进行通信

客户端将 MCP 服务器作为子进程启动。通信消息是JSON-RPC



SSE http：Server作为独立进程运行

Server功能

Tools： 与外部或者本地系统交互的工具

Prompts:提供Prompt模板

resources:额外资源(text,image之外的资源，目前没什么用)

Utilities：额外工具

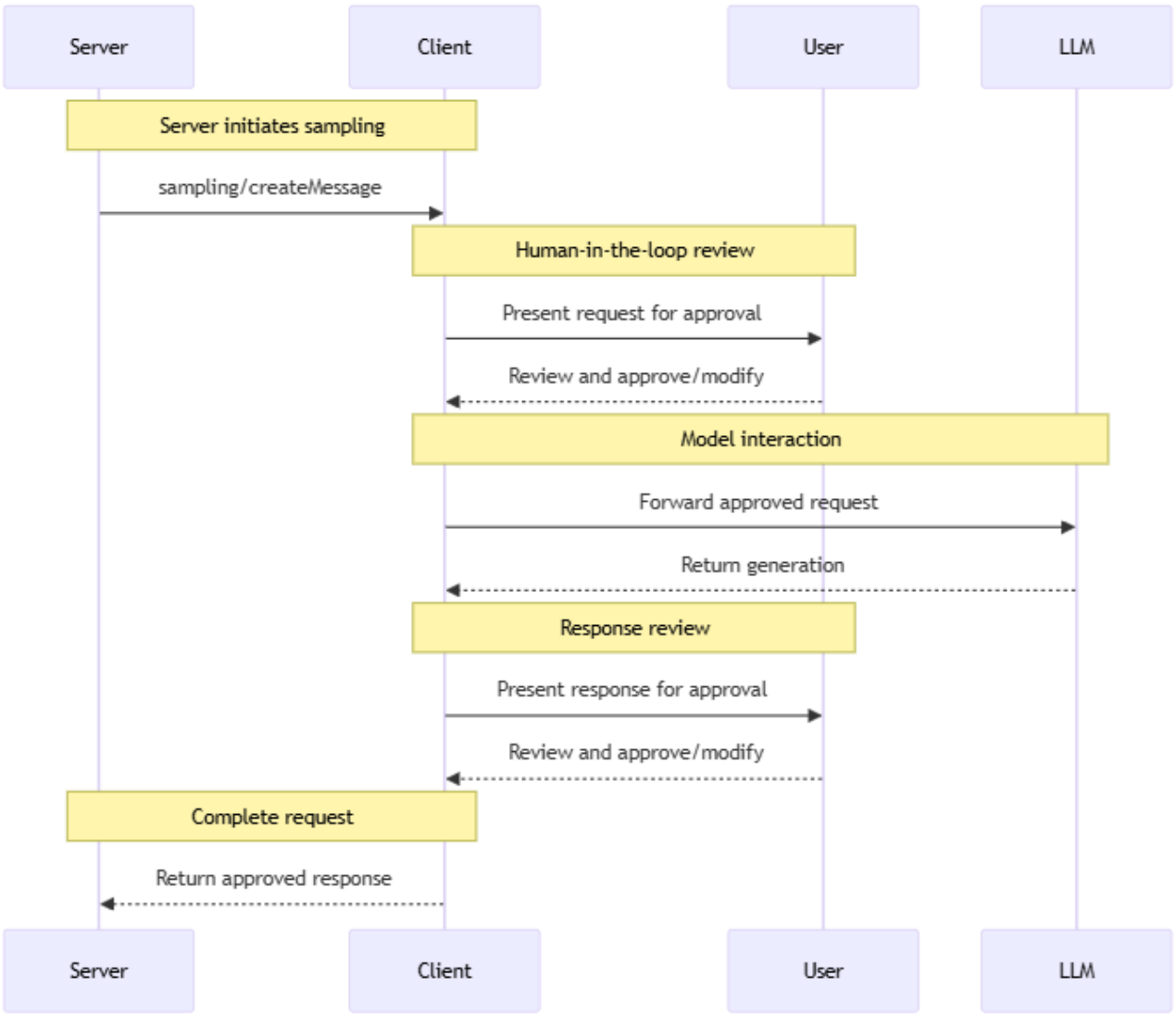
Client功能：

Roots：本地文件管理及权限管理

Sampling:MCP Server 借用本地LLM

Server--> Client->User ->LLM

Message Flow



编辑器功能支持

Example Clients - Model Context Protocol

Feature support matrix

Client	Resources	Prompts	Tools	Sampling	Roots	Notes
Claude Desktop App	✅	✅	✅	❌	❌	Full support for all MCP features
5ire	❌	❌	✅	❌	❌	Supports tools.
BeeAI Framework	❌	❌	✅	❌	❌	Supports tools in agentic workflows.
Cline	✅	❌	✅	❌	❌	Supports tools and resources.
Continue	✅	✅	✅	❌	❌	Full support for all MCP features
Cursor	❌	❌	✅	❌	❌	Supports tools.
Emacs Mcp	❌	❌	✅	❌	❌	Supports tools in Emacs.
Firebase Genkit	⚠️	✅	✅	❌	❌	Supports resource list and lookup through tools.
GenAIScript	❌	❌	✅	❌	❌	Supports tools.
Goose	❌	❌	✅	❌	❌	Supports tools.
LibreChat	❌	❌	✅	❌	❌	Supports tools for Agents
mcp-agent	❌	❌	✅	⚠️	❌	Supports tools, server connection management, and agent workflows.
Roo Code	✅	❌	✅	❌	❌	Supports tools and resources.
Sourcegraph Cody	✅	❌	❌	❌	❌	Supports resources through OpenCTX
Superinterface	❌	❌	✅	❌	❌	Supports tools
TheiaAI/TheiaIDE	❌	❌	✅	❌	❌	Supports tools for Agents in Theia AI and the AI-powered Theia IDE
Windsurf Editor	❌	❌	✅	❌	❌	Supports tools with AI Flow for collaborative development.
Zed	❌	✅	❌	❌	❌	Prompts appear as slash commands
SpinAI	❌	❌	✅	❌	❌	Supports tools for Typescript AI Agents
OpenSumi	❌	❌	✅	❌	❌	Supports tools in OpenSumi
Daydreams Agents	✅	✅	✅	❌	❌	Support for drop in Servers to Daydreams agents